

Task : Identify IDOR Flaw in the Profile Page

Name : Anjelina Behera

Course : CyberSecurity

Date : 9-08-26

Objective

The objective of this assignment was to identify an Insecure Direct Object Reference (IDOR) vulnerability in the profile page of a web application, manipulate the identifier parameter to access another user's profile, and retrieve sensitive information including the administrative user's flag.

Introduction

This assignment focused on understanding how IDOR vulnerabilities arise when applications expose internal object references such as database identifiers through URL parameters or request bodies without proper authorization checks. By examining the profile page functionality, intercepting HTTP requests with Burp Suite, and modifying the user identifier parameter, I was able to demonstrate unauthorized access to another user's private profile data. The exercise highlighted why direct object references must always be validated against the authenticated user's permissions on the server side.

Tools Used

- * Kali Linux - penetration testing operating system
- * Burp Suite Community Edition - HTTP proxy and request inspection
- * Web Browser - accessing the target web application
- * Browser Developer Tools - viewing page source and HTML
- * TryHackMe target machine - vulnerable web application

Prerequisites

What is IDOR?

Insecure Direct Object Reference (IDOR) is an access control vulnerability that occurs when an application exposes a reference to an internal implementation object, such as a database key or file path, and allows users to directly manipulate that reference to access unauthorized data. If the server does not verify that the authenticated user is authorized to access the requested object, an attacker can enumerate identifiers and view or modify other users' data.

What are Direct Object References?

Direct object references are identifiers used by the application to retrieve specific records from a database or filesystem. Examples include numeric user IDs, employee serial numbers, or document filenames passed through URL parameters, form fields, or API endpoints. When these references are predictable and not protected by authorization checks, they become a vector for unauthorized data access.

What is Access Control?

Access control ensures that users can only interact with resources they are explicitly permitted to access. In the context of IDOR, access control means verifying on the server side that the object being requested belongs to the currently authenticated user or that the user has explicit permission to access it. Relying solely on client-side controls or assuming that users will not tamper with parameters is insufficient.

Methodology

Step 1 - Authenticating to the Application

I logged into the web application using valid credentials. After authentication, I was redirected to the employee dashboard. From there, I navigated to the Profile section to view my own account information.

Step 2 - Observing the Profile Page URL

Upon loading the profile page, I examined the browser address bar and noticed that the URL contained a query parameter: `/profile.php?id=` followed by an identifier. This suggested that the application was using a direct object reference in the URL to determine which user's profile data to display. I noted my own identifier for comparison purposes.

Step 3 - Intercepting the Profile Request with Burp Suite

I configured Burp Suite as the browser proxy and reloaded the profile page with Intercept enabled. The captured GET request showed the `id` parameter being sent to the server. I sent this request to Burp Suite Repeater so that I could modify the parameter value and observe how the server responded to different identifiers.

Step 4 - Enumerating User Identifiers

In Burp Suite Repeater, I began modifying the `id` parameter to test whether the application performed any authorization checks. I tried incrementing the identifier, changing it to common patterns, and eventually substituting it with an identifier that appeared to belong to an administrative user. When I sent a request with the admin identifier, the server returned a profile page belonging to Akshay Admin, the CEO, instead of my own profile.

Step 5 - Accessing the Admin Profile

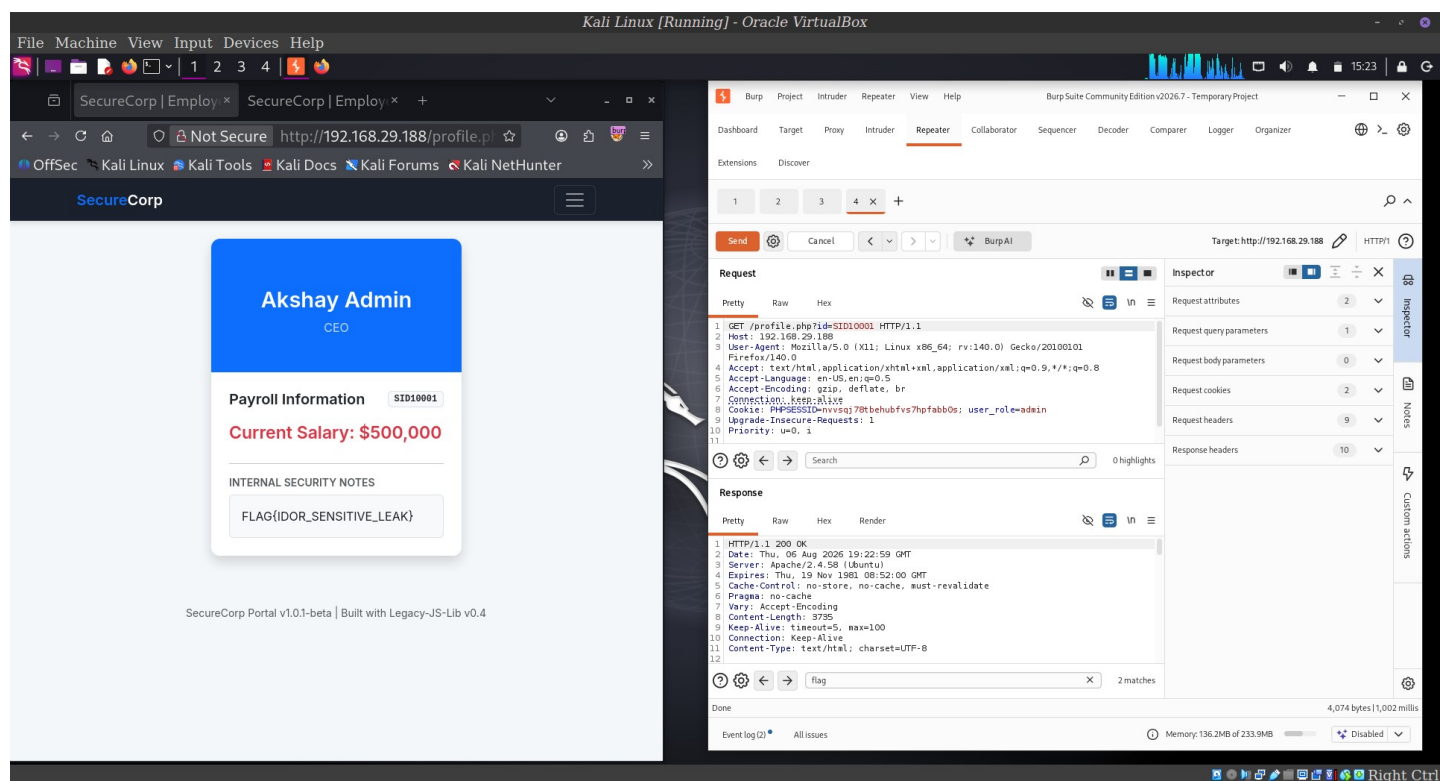
The response from the server contained the full profile page of the administrative user. It displayed sensitive information that was not visible on my own profile, including the admin's role as CEO, payroll information with a current salary, and internal security notes. The application had served this data without verifying whether the requesting user was authorized to view another employee's profile.

Step 6 - Locating the Flag

I examined the administrative profile page content and the corresponding HTTP response in Burp Suite. Within the internal security notes section of the profile, I found the assignment flag. This confirmed that the IDOR vulnerability had successfully exposed sensitive data intended only for the admin user.

Step 7 - Screenshot Evidence

I captured screenshots showing the Burp Suite Repeater with the modified id parameter, the HTTP response containing the admin profile data, and the browser rendering the unauthorized profile page with the flag visible in the internal security notes section.



Security Risks Identified

1 - Insecure Direct Object Reference (IDOR)

The profile endpoint accepted a user identifier from the client without verifying that the authenticated session was authorized to access the requested profile. This allowed any authenticated user to view any other user's profile by simply changing the id parameter. Risk: Unauthorized access to sensitive personal and financial data of other users.

2 - Lack of Server-Side Authorization Checks

The application failed to implement server-side authorization before serving profile data. It relied on the assumption that users would only access their own profiles through the provided interface, without enforcing this restriction at the API level. Risk: Data breaches, privacy violations, and exposure of high-value targets such as executives and administrators.

3 - Exposure of Sensitive Administrative Data

The admin profile contained confidential information including salary details, internal security notes, and flags. When combined with the IDOR vulnerability, this information became accessible to any authenticated user. Risk: Corporate espionage, targeted social engineering, and compromise of internal security procedures.

4 - Predictable Identifiers

The user identifiers followed a predictable pattern, making enumeration straightforward. An attacker could systematically iterate through identifiers to harvest the entire user database. Risk: Mass data exfiltration and complete compromise of user privacy across the application.

Real-World Relevance

IDOR vulnerabilities are among the most commonly reported and exploited weaknesses in modern web applications. They frequently appear in APIs, profile pages, document viewers, and order management systems. Real-world breaches have demonstrated that attackers can exploit IDOR to access millions of user records, modify account details, and escalate privileges. The impact is particularly severe when administrative accounts are exposed, as it can lead to full application compromise.

Organizations must implement indirect object references such as randomized tokens or UUIDs, enforce strict server-side authorization checks for every request that accesses user-specific data, and conduct regular access control testing as part of their security assessment programs.

Conclusion

This assignment provided practical experience in identifying and exploiting an IDOR vulnerability within a profile page. By observing the direct object reference in the URL, intercepting the request with Burp Suite, and modifying the identifier parameter, I successfully accessed the administrative user's profile and retrieved sensitive data including the assignment flag. The exercise demonstrated that even simple parameter tampering can have serious consequences when server-side authorization is missing. It reinforced the critical importance of validating every request against the authenticated user's permissions and avoiding predictable direct references to internal objects.